Software Requirements Specification

for

Smart Expense & Budget Management Application

Version 1.0 approved

Prepared by:

Shriram Chandrasekar (PES2UG24CS495)
Shreya S Mattimadu (PES2UG24CS486)
Shashwath K R (PES2UG24CS463)
Shashank A (PES2UG24CS461)

PES University

05 September 2026

Revision History

Name	Date	Reason For Changes	Version
Project Team	05-Sep-2026	Initial version	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of the Smart Expense & Budget Management Application, a web-based system for tracking personal expenses, managing budgets, and generating spending analytics and reports. This SRS covers the complete system as proposed by the project team.

1.2 Intended Audience and Reading Suggestions

This document is intended for the project's developers, project supervisors/evaluators, and testers. Sections 1-2 give an overview of the product and its context; Section 3 covers external interfaces; Section 5 details functional requirements organized by system feature; Section 6 covers non-functional requirements. Readers evaluating the project should focus on Sections 2, 5, and 6.

1.3 Product Scope

The Smart Expense & Budget Management Application allows individual users (and optionally households through a shared-budget mode) to record and categorize expenses, set budgets, receive alerts when spending crosses defined thresholds, and view analytics and exportable reports on their spending. The goal is to give users a clear, real-time picture of their finances and help them stay within their budgets.

1.4 References

Smart Expense & Budget Management Application - Functional & Non-Functional Requirements document (project team, 2026).

2. Overall Description

2.1 Product Perspective

This is a new, self-contained application and is not a follow-on to an existing product. It will be built as a web application, with the proposed technology stack of either Python (Django) or JavaScript (MERN stack), to be finalized by the team.

2.2 Product Functions

At a high level, the system will let users:

- Manage their account (registration, login, profile settings)
- Track individual and recurring expenses, including receipts and bulk CSV import
- Organize expenses using default or custom categories, with auto-suggested categorization
- Set and monitor budgets per category and overall, with real-time utilization tracking
- View analytics and predictive insights on their spending
- Receive alerts for budget thresholds and unusual transactions
- Generate and export spending reports

2.3 User Classes and Characteristics

The primary user class is an individual managing personal finances, with no special technical expertise required. Where the shared-budget mode is implemented, a secondary 'admin' user class manages shared budgets and membership, while 'member' users have restricted access to shared budget data. All user classes are expected to access the system through a standard web (and optionally mobile) browser.

2.4 Operating Environment

The application will run as a web-based client-server system, accessible via modern web browsers on desktop, tablet, and mobile devices.

2.5 Design and Implementation Constraints

The team must finalize a single technology stack (Python/Django or JavaScript/MERN) before implementation begins. Sensitive data must be encrypted and authentication must use JWT or session tokens. The system must follow a modular (MVC) architecture so that features such as alerts and analytics can be updated independently.

2.6 Assumptions and Dependencies

The project assumes access to standard third-party or open-source libraries for charting, PDF/Excel export, and (optionally) OAuth/Google login. Bulk import assumes users can supply expense data in CSV or standard bank-statement formats. Automated categorization assumes a rule-based approach is sufficient initially, with a lightweight ML classifier as a possible enhancement.

3. External Interface Requirements

3.1 User Interfaces

The system will provide a responsive web interface with screens for login/registration, an expense entry form, a category manager, a budget dashboard showing utilization per category, an analytics/insights view with charts, an alerts/notifications panel, and a reports/export screen. The interface will follow WCAG accessibility basics (sufficient contrast, labeled form fields).

3.2 Software Interfaces

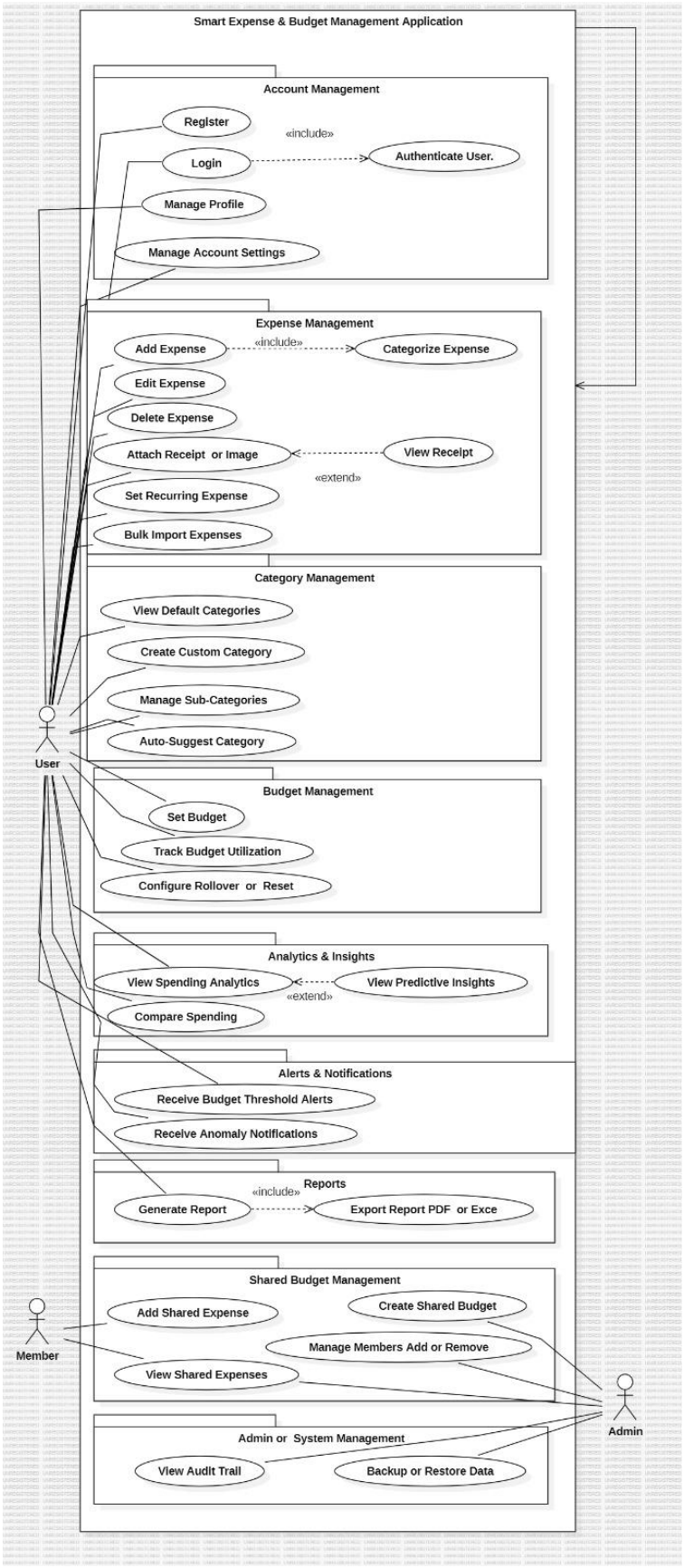
The system will interface with a relational or document database (depending on the finalized stack) to store users, expenses, categories, and budgets; with an email service for delivering alerts; and, optionally, with an OAuth/Google identity provider for login.

3.3 Communications Interfaces

All client-server communication will use HTTPS/TLS. Email alerts will be sent using a standard transactional email protocol/service. The system will expose or consume a JSON-based API between the frontend and backend.

4. Analysis Models

Use case diagrams and an entity-relationship diagram covering Users, Expenses, Categories, Budgets, and Alerts will be developed during the design phase and included here in a future revision.


Smart Expense & Budget Management Application
Account Management
Register
«include»
Login
Authenticate User.
Manage Profile
Manage Account Settings
Expense Management
Add Expense
«include»
Categorize Expense
Edit Expense
Delete Expense
Attach Receipt or Image
View Receipt
«extend»
Set Recurring Expense
Bulk Import Expenses
Category Management
View Default Categories
Create Custom Category
Manage Sub-Categories
Auto-Suggest Category
User
Budget Management
Set Budget
Track Budget Utilization
Configure Rollover or Reset
Analytics & Insights
View Spending Analytics
View Predictive Insights
«extend»
Compare Spending
Alerts & Notifications
Receive Budget Threshold Alerts
Receive Anomaly Notifications
Reports
«include»
Generate Report
Export Report PDF or Exce
Shared Budget Management
Add Shared Expense
Create Shared Budget
Manage Members Add or Remove
View Shared Expenses
Member
Admin
Admin or System Management
View Audit Trail
Backup or Restore Data

5. System Features

5.1 User Management

5.1.1 Description and Priority

User Management lets the user register, log in, and manage their account and access level. Priority: High.

5.1.2 Stimulus/Response Sequences

User submits registration or login details; the system validates credentials and either grants access or returns an error message.

5.1.3 Functional Requirements

ID	Requirement
FR1	Users shall be able to register and log in using email/password (and optionally OAuth/Google login).
FR2	Users shall be able to update profile details and manage account settings.
FR3	System shall support role-based access if a multi-user/family/shared-budget mode is included (admin vs. member).

5.2 Expense Tracking

5.2.1 Description and Priority

Expense Tracking lets the user record, edit, delete, and import expenses. Priority: High.

5.2.2 Stimulus/Response Sequences

User submits a new expense (or a CSV file); the system validates and stores the entry, updating the relevant budget totals.

5.2.3 Functional Requirements

ID	Requirement
FR4	Users shall be able to add, edit, and delete individual expense entries (amount, date, payment mode, notes).
FR5	Users shall be able to attach a receipt/image to an expense.
FR6	System shall support recurring expense entries (e.g., rent, subscriptions).
FR7	System shall allow bulk import of expenses via CSV/bank statement upload.

5.3 Categories

5.3.1 Description and Priority

Categories lets the user organize expenses into default or custom categories. Priority: High.

5.3.2 Stimulus/Response Sequences

User creates or selects a category while entering an expense; the system may auto-suggest a category based on the merchant or past patterns.

5.3.3 Functional Requirements

ID	Requirement
FR8	System shall provide default expense categories (food, travel, bills, etc.) and allow users to create custom categories/sub-categories.
FR9	System shall auto-suggest a category for an entry based on past patterns or merchant name (rule-based or lightweight ML classifier).

5.4 Budget Management

5.4.1 Description and Priority

Budget Management lets the user set and track budgets. Priority: High.

5.4.2 Stimulus/Response Sequences

User sets a budget for a category or period; the system updates and displays remaining balance as new expenses are recorded.

5.4.3 Functional Requirements

ID	Requirement
FR10	Users shall be able to set monthly/weekly budgets overall and per category.
FR11	System shall track budget utilization in real time and show remaining balance per category.
FR12	System shall allow budget rollover or reset configuration at period end.

5.5 Analytics & Insights

5.5.1 Description and Priority

Analytics & Insights lets the user view visual analytics and projections of their spending. Priority: High.

5.5.2 Stimulus/Response Sequences

User opens the analytics view; the system renders charts and projections based on stored expense data.

5.5.3 Functional Requirements

ID	Requirement
FR13	System shall generate visual analytics (pie/bar/line charts) for spending by category, time period, and trend over months.
FR14	System shall provide predictive insights (e.g., projected month-end spend based on current rate).
FR15	System shall allow comparison of spending across months/years.

5.6 Alerts

5.6.1 Description and Priority

Alerts lets the user receive threshold and anomaly notifications. Priority: High.

5.6.2 Stimulus/Response Sequences

The system detects a budget threshold breach or an anomalous transaction and sends an in-app and/or email notification to the user.

5.6.3 Functional Requirements

ID	Requirement
FR16	System shall send alerts when spending crosses a threshold (e.g., 80%, 100% of category budget).
FR17	System shall notify users of unusual/anomalous transactions (spike detection).
FR18	Alerts shall be deliverable via in-app notification and email (push notification optional).

5.7 Reports

5.7.1 Description and Priority

Reports lets the user generate and export spending reports. Priority: High.

5.7.2 Stimulus/Response Sequences

User selects a date range and requests a report; the system compiles the data and returns a downloadable PDF/Excel file.

5.7.3 Functional Requirements

ID	Requirement
FR19	Users shall be able to generate and export reports (PDF/Excel) for a custom date range.
FR20	Reports shall include summary statistics, category-wise breakdown, and budget vs. actual comparison.

5.8 Admin / System

5.8.1 Description and Priority

Admin / System lets the user maintain data integrity through auditing and backup. Priority: High.

5.8.2 Stimulus/Response Sequences

The system logs each edit to expenses/budgets and performs scheduled backups; an admin can trigger a restore if needed.

5.8.3 Functional Requirements

ID	Requirement
FR21	System shall maintain an audit trail/history of edits to expenses and budgets.
FR22	System shall support data backup and restore for user records.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Requirement
NFR1	Dashboard and analytics pages shall load within 2-3 seconds for up to 10,000 transaction records per user.
NFR2	Report generation shall complete within 5 seconds for a 1-year data range.

6.2 Safety Requirements

Not applicable in the traditional sense, as this is a financial data management application rather than a safety-critical system. The primary safety concern is preventing data loss, addressed through the reliability and backup requirements in Section 6.4.

6.3 Security Requirements

ID	Requirement
NFR4	All sensitive data (passwords, financial info) shall be encrypted at rest and in transit (HTTPS/TLS, hashed passwords via bcrypt).
NFR5	System shall implement JWT/session-based authentication with token expiry.
NFR6	System shall be protected against common vulnerabilities (SQL injection, XSS, CSRF).

6.4 Software Quality Attributes

Scalability

ID	Requirement
NFR3	System architecture shall support horizontal scaling to accommodate growth in concurrent users.

Usability

ID	Requirement
NFR7	UI shall be responsive across mobile, tablet, and desktop.
NFR8	The interface shall follow accessibility guidelines (WCAG basics - contrast, labels).

Reliability & Availability

ID	Requirement
NFR9	System shall have a 99% uptime target for the deployed environment.
NFR10	Automated daily backups of the database shall be scheduled.

Maintainability

ID	Requirement
NFR11	Codebase shall follow modular architecture (MVC pattern) to allow independent updates to modules like alerts/analytics.
NFR12	System shall include logging (application logs + error logs) for debugging.

Portability

ID	Requirement
NFR13	Application shall be deployable on common cloud platforms (AWS/Azure/Heroku/Render) with containerization (Docker) support.

6.5 Business Rules

Only authenticated users may view or modify their own expense and budget data. In shared-budget mode, only an 'admin' member may add or remove members or change shared-budget settings; regular members may only add/view expenses within the shared budget.

7. Other Requirements

Compliance

ID	Requirement
NFR14	System shall comply with data privacy expectations for storing personal financial data (no third-party sharing without consent).

Appendix A: Glossary

- FR - Functional Requirement
- NFR - Non-Functional Requirement
- Category - A label (e.g., Food, Travel, Bills) used to classify an expense
- Budget - A spending limit set by the user for a category or time period
- MVC - Model-View-Controller architectural pattern
- JWT - JSON Web Token, used for authentication

Appendix B: Field Layouts

Detailed field layouts (field name, length, data type, description, mandatory flag) for entities such as Expense, Category, and Budget will be documented in a separate data-dictionary spreadsheet as the database design is finalized.

Appendix C: Requirement Traceability Matrix

A full traceability matrix mapping each requirement ID to its architecture reference, design reference, code file, and test case ID will be maintained separately and updated as design and implementation progress.